UNIVERSIDADE ESTADUAL DA PARAÍBA – UEPB
CENTRO DE CIÊNCIAS E TECNOLOGIA – CCT
DEPARTAMENTO DE COMPUTAÇÃO
[LABORATÓRIO DE] ESTRUTURA DE DADOS – 2026.2

Análise de Algoritmos de Busca e Ordenação em Java

Relatório da Atividade da Unidade 1

ARTHUR RONALD SOARES EVARISTO
KARLOS EDUARDO SARAIVA DA SILVA

Professor: JANDERSON JASON BARBOSA AGUIAR

Campina Grande – PB, outubro de 2026

1. Introdução

Este relatório apresenta a implementação e a análise experimental de algoritmos de busca e de ordenação em Java, trabalhando exclusivamente com vetores de inteiros (`int[]`) com valores no intervalo [1, 100.000], com repetições permitidas. O objetivo é confrontar a **teoria** (complexidade assintótica) com a **prática** (tempos medidos), identificando em quais cenários cada algoritmo é mais vantajoso e explicando as divergências observadas.

Foram implementados e comparados os seguintes algoritmos de ordenação: BubbleSort (versão do slide e versão otimizada), SelectionSort (versão do slide e versão estável), InsertionSort, MergeSort (versão clássica), QuickSort (versão do slide e versão do slide com embaralhamento prévio) e CountingSort. O método `Arrays.sort(int[])` da biblioteca padrão foi usado como referência. Para busca, foram implementadas a busca linear iterativa, a linear recursiva, a linear iterativa de duas pontas, a binária iterativa e a binária recursiva.

Todo o código-fonte, os dados brutos de cada execução e os gráficos estão disponíveis no repositório: https://github.com/ArthurRonald/java-sorting-benchmark.
O vídeo de apresentação do trabalho está disponível em: https://youtu.be/d4RyaPR6Onw.

2. Metodologia

2.1 Ambiente de execução

Todos os experimentos foram executados em um único computador, para que os tempos fossem comparáveis entre si:

- Sistema operacional: Windows 11 (versão 10.0, arquitetura amd64);
- Processador: Intel Core i5-12450HX (8 núcleos: 4 de alto desempenho e 4 de eficiência; 12 threads), com 12 núcleos lógicos disponíveis para a JVM;
- Memória RAM: 16 GB DDR5; memória máxima da JVM: 4.028 MB;
- Placa de vídeo: NVIDIA GeForce RTX 3050 Laptop (6 GB), não utilizada nos experimentos, que rodam inteiramente na CPU;
- Java: 26.0.1 (Java HotSpot 64-Bit Server VM);
- Condições de execução: Computador ligado na tomada e sem programas ativos em primeiro plano durante a execução.

O experimento principal de ordenação levou cerca de 40 minutos e o de vetores grandes, cerca de 13 minutos (registrado nos arquivos `resultados/log_*.txt`).

2.2 Implementações

Os algoritmos seguem os pseudocódigos dos slides da disciplina, com os índices ajustados de 1..N para 0..N−1. Detalhes relevantes de cada versão:

- **BubbleSort otimizado:** interrompe a ordenação quando uma passada inteira não realiza nenhuma troca.
- **SelectionSort estável:** em vez de trocar o menor elemento com o da posição i, desloca os elementos entre i e a posição do menor uma casa para a direita e insere o menor em i; a busca do mínimo usa "<" para manter a primeira ocorrência.
- **MergeSort:** versão clássica top-down recursiva; o vetor auxiliar de tamanho n é alocado uma única vez; o merge usa "<=", o que garante a estabilidade.
- **QuickSort (slide):** partição "à la Hoare" do slide, com pivô fixo no primeiro elemento (`A[left]`).
- **QuickSort + shuffle:** embaralha o vetor com o algoritmo de Fisher-Yates e depois aplica o QuickSort do slide; o tempo do embaralhamento faz parte da medição.
- **CountingSort:** versão do slide com k = 100.000 (maior valor possível); percorre a entrada de trás para frente (estável) e copia o vetor de saída de volta para o vetor original.
- **Buscas:** todas retornam o índice do elemento ou −1; as versões recursivas usam um método auxiliar com os índices.

2.3 Entradas e cenários

Os vetores foram gerados com valores inteiros uniformes em [1, 100.000], usando sementes fixas (os experimentos são reproduzíveis). Foram avaliados quatro cenários:

- **Aleatório:** valores sorteados uniformemente;
- **Já ordenado:** o vetor aleatório em ordem crescente;
- **Ordem inversa:** o vetor ordenado em ordem decrescente;
- **Parcialmente ordenado:** a partir do vetor ordenado, cerca de 15% das posições são sorteadas e apenas os elementos dessas posições são embaralhados entre si.

Para as buscas, o vetor é previamente ordenado e o elemento procurado é sempre −1, que não pertence ao intervalo de valores. Isso força o pior caso de todas as buscas.

2.4 Tamanhos dos vetores

- **Busca:** n = 50.000, 150.000 e 500.000.
- **Ordenação, experimento principal (os 10 algoritmos):** n = 25.000, 50.000 e 100.000.
- **Ordenação, vetores grandes (MergeSort, QuickSort, QuickSort + shuffle, CountingSort e Arrays.sort):** n = 50.000, 150.000 e 500.000.

Justificativa: o enunciado sugere 50.000, 150.000 e 500.000 elementos, mas permite ajustar os tamanhos quando o tempo de execução inviabiliza os experimentos, desde que algoritmos comparados diretamente usem os mesmos tamanhos. Em medições preliminares (feitas em outro computador, apenas para dimensionar o experimento), o BubbleSort em ordem inversa levou cerca de 10 ms com n = 4.000. Como ele é $\Theta(n^2)$, com n = 500.000 cada execução levaria minutos, e as 25 execuções × 4 cenários × 2 versões somariam várias horas. Por isso os 10 algoritmos foram

comparados com os mesmos tamanhos (25k, 50k e 100k, cada um o dobro do anterior), e os algoritmos eficientes foram avaliados também nos tamanhos sugeridos pelo professor.

2.5 Procedimento de medição

- O tempo foi medido com `System.nanoTime()` imediatamente antes e depois da chamada do algoritmo.
- Cada combinação (algoritmo × tamanho × cenário) foi executada **25 vezes**; as **5 primeiras** foram descartadas como *warm-up* da JVM (período em que o compilador JIT ainda está otimizando o código) e as estatísticas usam as 20 restantes.
- Para cada par (tamanho, cenário) foi gerado **um único vetor original**, e cada execução de cada algoritmo recebeu uma **cópia** dele. Nenhum algoritmo recebeu como entrada a saída de outro.
- Foram calculados média, desvio padrão amostral, mediana, mínimo e máximo. As tabelas da seção 4 apresentam a média; a discussão usa também a mediana, que é menos sensível a valores extremos.
- **Buscas:** como uma busca binária dura poucas dezenas de nanossegundos, valor próximo da resolução do relógio, cada execução mediu um lote de 1.000 buscas consecutivas, e o valor reportado é o tempo médio de uma busca.
- **Tempo limite de segurança:** se uma única execução passasse de 20 s, a combinação era interrompida e registrada como tal (seção 4.4).
- O programa é executado em uma *thread* com pilha de 1 GB, pois a busca linear recursiva e o QuickSort no pior caso atingem profundidades de recursão da ordem de n, o que causaria `StackOverflowError` com a pilha padrão da JVM.

2.6 Verificação de correção

Após cada execução, e fora do intervalo medido, o resultado foi verificado de duas formas: (i) o vetor está em ordem não decrescente; e (ii) é idêntico ao resultado de referência (`Arrays.sort` aplicado a uma cópia da entrada), o que garante que nenhum elemento foi perdido ou duplicado. Nas buscas, verificou-se que todas as chamadas retornaram −1. Antes dos experimentos, um modo de teste executou todos os algoritmos em 630 casos de ordenação e 5.170 casos de busca, incluindo casos-limite (vetor vazio, um elemento, todos os elementos iguais, valores extremos). Nenhuma verificação falhou.

3. Complexidade esperada

Algoritmo	Melhor caso	Caso médio	Pior caso	Memória extra	Estável
BubbleSort (slide)	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Sim
BubbleSort otimizado	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Sim
SelectionSort (slide)	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Não
SelectionSort estável	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Sim
InsertionSort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Sim
MergeSort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(n)$	Sim
QuickSort (slide)	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$O(\log n)$ médio; $O(n)$ pior	Não
QuickSort + shuffle	$\Theta(n \log n)$	$\Theta(n \log n)$ esperado	$\Theta(n^2)$, improvável	$O(\log n)$ esperado	Não
CountingSort	$\Theta(n + k)$	$\Theta(n + k)$	$\Theta(n + k)$	$O(n + k)$	Sim
Arrays.sort(int[])	—	$O(n \log n)$	$O(n \log n)$	$O(\log n)$	n/a (primitivos)

Tabela 1: complexidade teórica dos algoritmos de ordenação ($k = 100.000$).

O MergeSort satisfaz $T(n) = 2T(n/2) + \Theta(n)$, o que pelo método mestre (caso 2) resulta em $\Theta(n \log n)$. O QuickSort tem o mesmo comportamento quando as partições são equilibradas; quando o pivô é sempre o menor ou o maior elemento, $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$. `Arrays.sort(int[])` usa um Dual-Pivot QuickSort com otimizações, inclusive a detecção de trechos já ordenados.

Busca	Pior caso (elemento ausente)	Recorrência
Linear iterativa	$\Theta(n)$	—
Linear recursiva	$\Theta(n)$	$T(n) = T(n-1) + \Theta(1)$
Linear duas pontas	$\Theta(n)$, com ~n/2 iterações	—
Binária iterativa	$\Theta(\log n)$	—
Binária recursiva	$\Theta(\log n)$	$T(n) = T(n/2) + \Theta(1)$

Tabela 2: complexidade teórica dos algoritmos de busca.

4. Resultados

As tabelas apresentam a **média das 20 execuções válidas**. Desvio padrão, mediana, mínimo e máximo de cada combinação estão em `resultados/*_resumo.csv`, e os tempos de todas as execuções em `resultados/*_brutos.csv`.

4.1 Busca

n	Linear iterativa	Linear recursiva	Linear duas pontas	Binária iterativa	Binária recursiva
50.000	7.509,1	133.670,6	11.785,4	25,5	32,0
150.000	34.138,4	352.436,3	36.394,2	18,5	25,2
500.000	105.261,4	4.462.865,4	128.155,6	20,7	25,7

Tabela 3: tempo médio por busca (ns), elemento procurado −1.

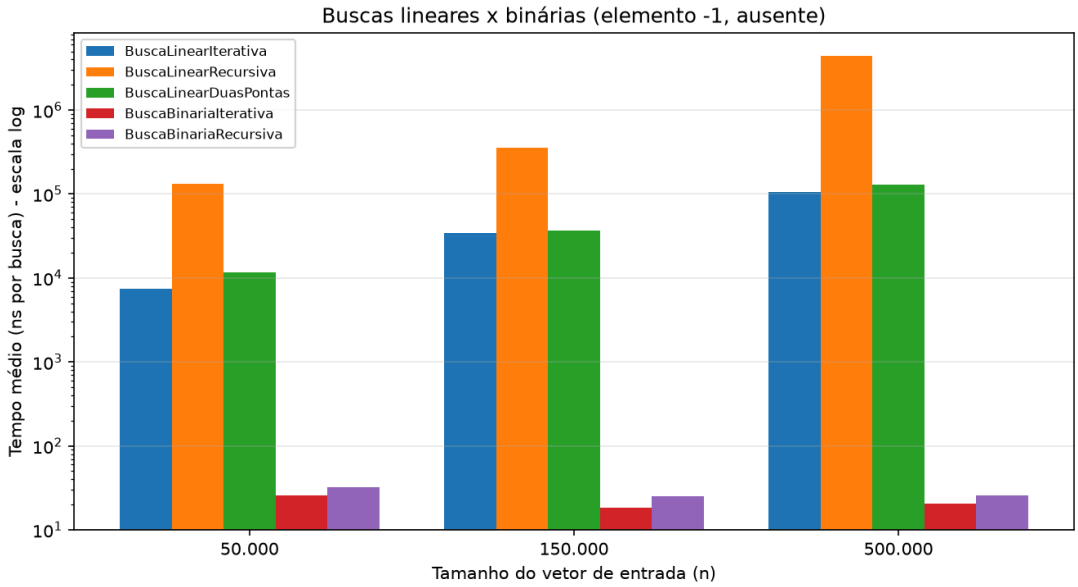


Figura 1: tempo médio por busca, buscas lineares × binárias (eixo y em escala logarítmica).

4.2 Ordenação: experimento principal (10 algoritmos)

Algoritmo	n = 25.000	n = 50.000	n = 100.000
BubbleSort (slide)	885,98	3.011,15	interrompido[1]
BubbleSort otimizado	677,13	2.964,14	interrompido[1]
SelectionSort (slide)	187,54	449,30	2.771,86
SelectionSort estável	329,46	388,77	3.378,38
InsertionSort	68,03	114,83	603,03
MergeSort	3,49	5,99	16,65
QuickSort (slide)	2,06	3,43	10,67
QuickSort + shuffle	2,27	3,44	12,18
CountingSort	0,902	0,349	1,34
Arrays.sort	2,93	2,50	9,42

[1] Interrompido pelo tempo limite de 20 s por execução (seção 4.4).

Tabela 4: tempo médio (ms), cenário aleatório.

Algoritmo	n = 25.000	n = 50.000	n = 100.000
BubbleSort (slide)	243,54	303,07	3.866,24
BubbleSort otimizado	0,181	0,011	0,085
SelectionSort (slide)	322,15	436,57	1.779,70
SelectionSort estável	259,82	365,16	1.448,68
InsertionSort	0,033	0,030	0,064
MergeSort	1,40	1,65	3,05
QuickSort (slide)	525,10	662,26	2.121,76
QuickSort + shuffle	3,29	4,38	7,21
CountingSort	0,668	0,198	0,796
Arrays.sort	0,156	0,008	0,046

Tabela 5: tempo médio (ms), cenário já ordenado.

Algoritmo	n = 25.000	n = 50.000	n = 100.000
BubbleSort (slide)	262,07	1.649,86	6.527,60
BubbleSort otimizado	275,88	1.576,83	6.323,61
SelectionSort (slide)	371,74	1.511,86	7.993,85
SelectionSort estável	176,49	1.688,94	12.436,45
InsertionSort	60,67	215,30	844,13
MergeSort	1,64	1,49	3,79
QuickSort (slide)	161,34	579,05	1.812,78
QuickSort + shuffle	1,74	4,60	7,23
CountingSort	0,231	0,406	0,364
Arrays.sort	0,155	0,098	0,062

Tabela 6: tempo médio (ms), cenário ordem inversa.

Algoritmo	n = 25.000	n = 50.000	n = 100.000
BubbleSort (slide)	172,06	797,01	3.302,99
BubbleSort otimizado	174,12	887,71	3.265,32
SelectionSort (slide)	117,53	537,66	1.743,53
SelectionSort estável	101,43	516,11	1.453,87
InsertionSort	4,61	28,60	78,18
MergeSort	0,922	2,22	4,40
QuickSort (slide)	1,82	7,00	5,90
QuickSort + shuffle	2,42	3,95	8,28
CountingSort	0,199	0,246	0,374
Arrays.sort	0,680	1,31	2,95

Tabela 7: tempo médio (ms), cenário parcialmente ordenado (15%).

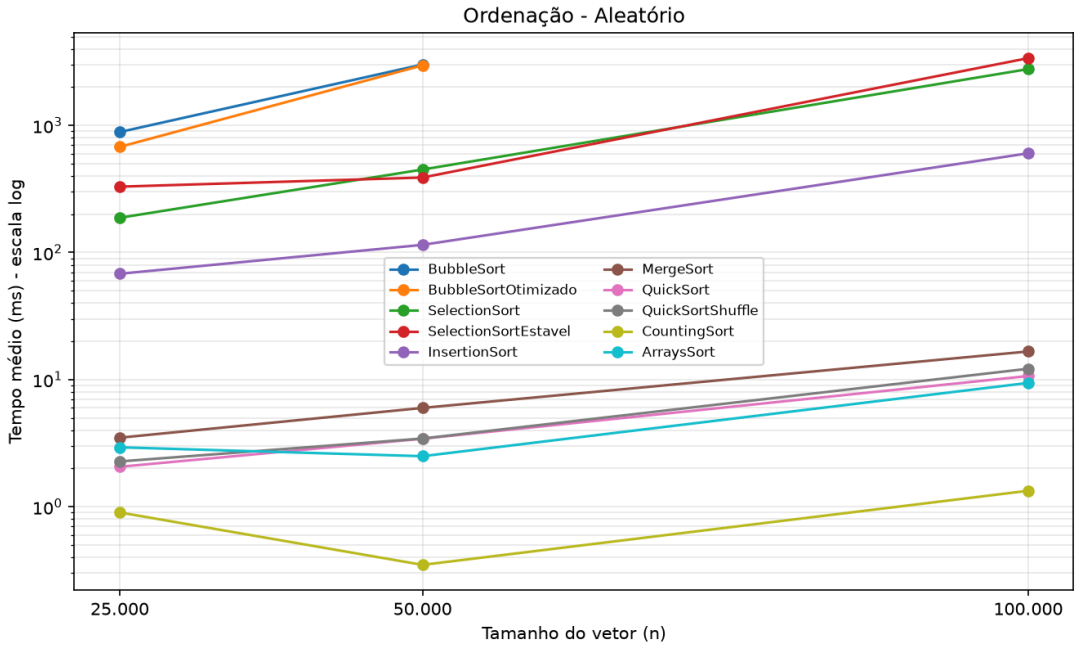


Figura 2: tempo × n no cenário aleatório (escala logarítmica).

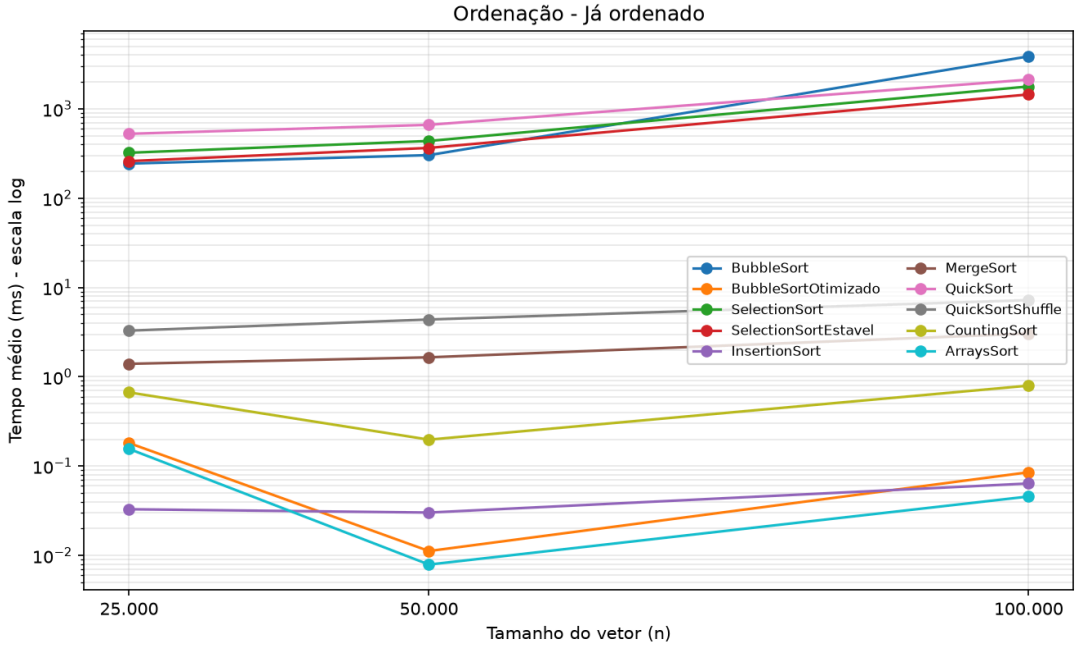


Figura 3: tempo × n no cenário já ordenado (escala logarítmica).

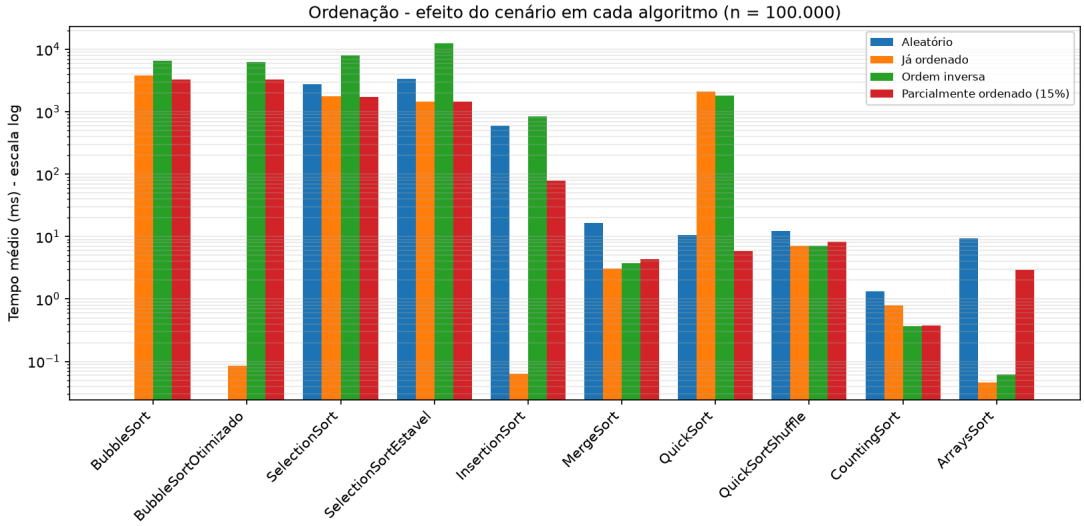


Figura 4: efeito do cenário de entrada em cada algoritmo, n = 100.000 (escala logarítmica).

4.3 Ordenação: vetores grandes (algoritmos eficientes)

Algoritmo	n = 50.000	n = 150.000	n = 500.000
MergeSort	4,94	33,76	54,89
QuickSort (slide)	3,40	18,34	43,18
QuickSort + shuffle	6,41	18,78	48,18
CountingSort	1,30	2,41	3,12
Arrays.sort	3,58	17,43	40,21

Tabela 8: tempo médio (ms) com vetores grandes, cenário aleatório.

Algoritmo	n = 50.000	n = 150.000	n = 500.000
MergeSort	1,69	11,38	18,57
QuickSort (slide)	677,33	6.901,37	16.038,75
QuickSort + shuffle	3,50	13,68	63,35
CountingSort	0,192	0,649	7,65
Arrays.sort	0,060	0,032	0,166

Tabela 9: tempo médio (ms) com vetores grandes, cenário já ordenado.

Algoritmo	n = 50.000	n = 150.000	n = 500.000
MergeSort	1,61	5,18	26,46
QuickSort (slide)	580,95	3.563,09	interrompido[1]
QuickSort + shuffle	8,14	14,77	56,43
CountingSort	0,332	0,580	2,37
Arrays.sort	0,240	0,163	0,478

[1] Interrompido pelo tempo limite de 20 s por execução (seção 4.4).

Tabela 10: tempo médio (ms) com vetores grandes, cenário ordem inversa.

Algoritmo	n = 50.000	n = 150.000	n = 500.000
MergeSort	4,95	7,57	27,08
QuickSort (slide)	7,91	10,82	39,00
QuickSort + shuffle	5,34	11,63	48,82
CountingSort	0,293	0,449	5,00
Arrays.sort	3,73	5,29	16,16

Tabela 11: tempo médio (ms) com vetores grandes, cenário parcialmente ordenado (15%).

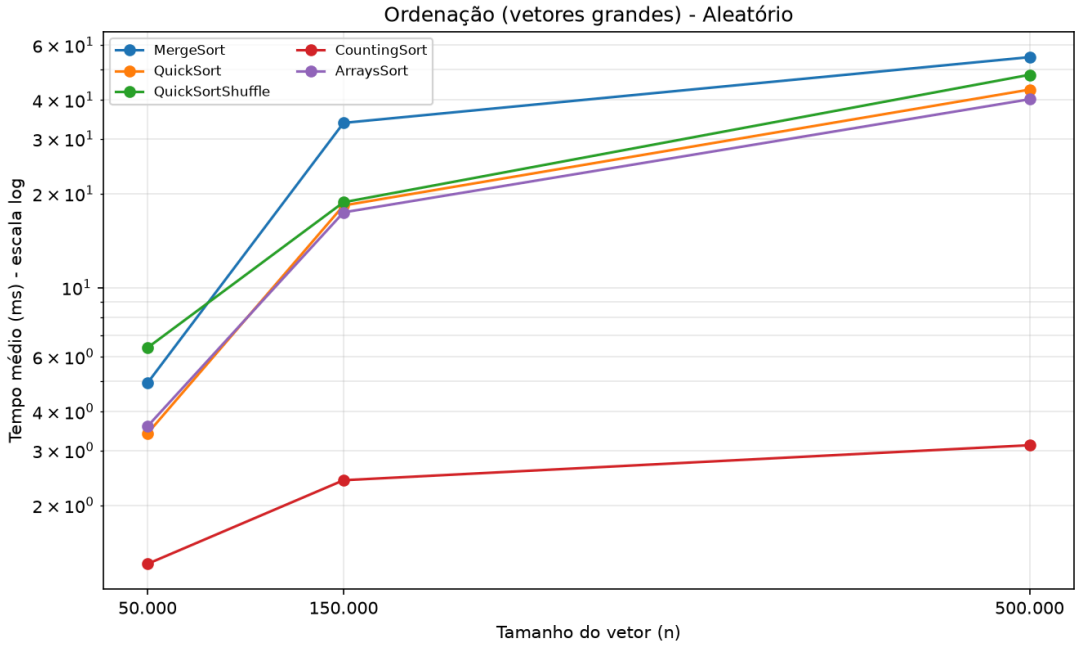


Figura 5: tempo × n com vetores grandes, cenário aleatório (escala logarítmica).

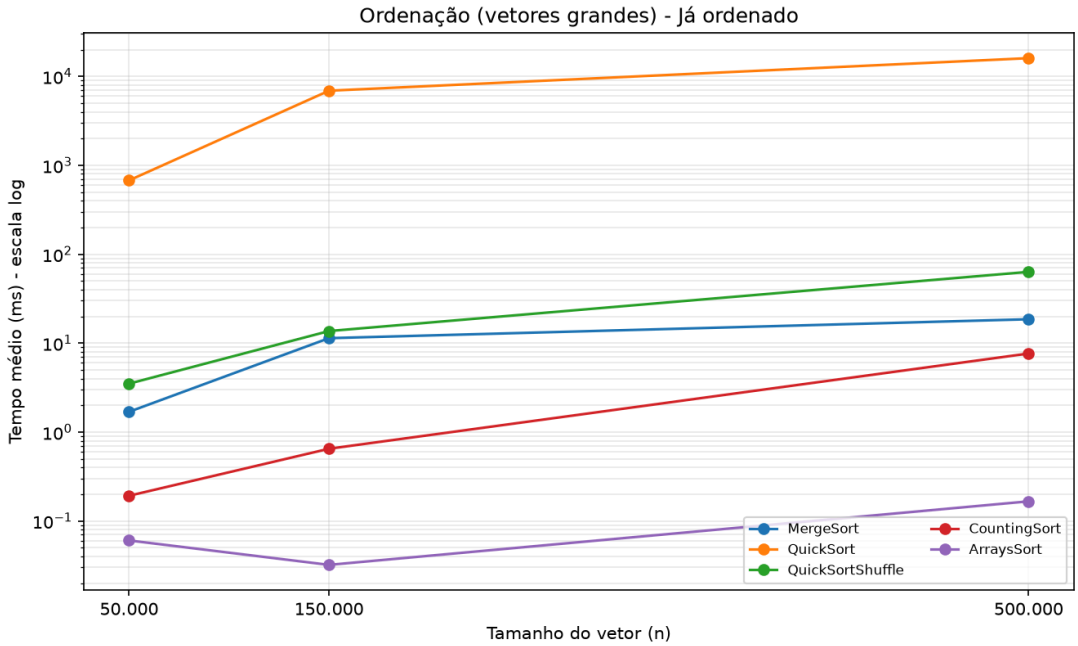


Figura 6: tempo × n com vetores grandes, cenário já ordenado (escala logarítmica).

4.4 Combinações interrompidas e variabilidade das medições

Algoritmo	Cenário	n	O que aconteceu (fonte: *_brutos.csv)
BubbleSort (slide)	aleatório	100.000	A 1ª execução (warm-up) levou 20,08 s; a combinação foi interrompida sem execuções válidas.
BubbleSort otimizado	aleatório	100.000	Os 5 warm-ups levaram entre 12,35 e 13,93 s; das execuções válidas, 4 ficaram entre 12,88 e 19,49 s e a 5ª levou 43,32 s. Com apenas 5 execuções válidas (abaixo das 20 exigidas), a média não foi reportada.
QuickSort (slide)	inverso	500.000	A 1ª execução (warm-up) levou 25,27 s.

Tabela 12: combinações interrompidas pelo tempo limite de 20 s por execução.

No caso do BubbleSort, a interrupção é coerente com $\Theta(n^2)$: de 25.000 para 50.000 o tempo médio cresceu ≈3,4× (885,98 → 3.011,15 ms), e a projeção para 100.000 (mais ≈4×) é de cerca de 12 s por execução, valor de fato observado nas execuções do BubbleSort otimizado. Esse tempo ficou próximo do limite, e uma oscilação de desempenho (a execução de 43,32 s) fez o limite ser ultrapassado. Assim, a comparação dos algoritmos quadráticos no cenário aleatório com n = 100.000 fica restrita a SelectionSort, SelectionSort estável e InsertionSort. A interrupção do QuickSort (slide) em ordem inversa com n = 500.000 é o seu pior caso, discutido na seção 5.3.

Variabilidade. Algumas medições apresentaram variação maior que a esperada, provavelmente por interferência do sistema operacional, variação de frequência do processador (economia de energia e temperatura) ou atividade de outros programas. Exemplos: a execução de 43,32 s citada acima; o SelectionSort estável aleatório cresceu ×1,18 de 25k para 50k e ×8,69 de 50k para 100k, quando a teoria prevê ≈×4 nos dois casos; e a busca linear recursiva com n = 500.000 teve média de 4,46 ms com desvio padrão de 2,31 ms. Outro indício é que a mesma combinação, medida nos dois experimentos de ordenação (n = 50.000), produziu valores diferentes, como o QuickSort + shuffle aleatório (3,44 ms no principal e 6,41 ms no de vetores grandes). Por isso a análise a seguir usa a mediana nas razões de crescimento e foca em ordens de grandeza e tendências consistentes, não em diferenças pequenas.

5. Discussão: teoria × prática

5.1 Crescimento do tempo com n

Quando n dobra, a teoria prevê que o tempo seja multiplicado por ≈4 para algoritmos $\Theta(n^2)$, por ≈2,13 para $\Theta(n \log n)$ (de 50k para 100k) e por ≈2 para $\Theta(n)$. A Tabela 13 mostra as razões observadas (medianas, de 50.000 para 100.000).

Algoritmo	Cenário	Razão observada (50k → 100k)	Previsto
BubbleSort (slide)	inverso	3,93	4 (Θ(n²))
BubbleSort otimizado	inverso	4,14	4 (Θ(n²))
BubbleSort (slide)	parcialmente ordenado	4,26	4 (Θ(n²))
SelectionSort (slide)	já ordenado	4,12	4 (Θ(n²))
SelectionSort estável	já ordenado	4,03	4 (Θ(n²))
SelectionSort (slide)	aleatório	4,65	4 (Θ(n²))
InsertionSort	aleatório	4,33	4 (Θ(n²))
InsertionSort	inverso	4,13	4 (Θ(n²))
InsertionSort	já ordenado	2,01	2 (Θ(n), melhor caso)
MergeSort	aleatório	3,02	2,13 (Θ(n log n))
QuickSort + shuffle	aleatório	3,53	2,13 (Θ(n log n))
Arrays.sort	aleatório	3,88	2,13 (Θ(n log n))

Tabela 13: razões de crescimento observadas (medianas) × previstas pela teoria.

Os algoritmos quadráticos confirmaram a teoria com boa precisão: em vários casos a razão ficou entre 3,9 e 4,3. O InsertionSort no cenário ordenado dobrou de tempo ao dobrar n (2,01), exatamente o comportamento linear do seu melhor caso.

Para os algoritmos Θ(n log n), as razões entre 50k e 100k ficaram acima de 2,13. Como esses tempos são de apenas alguns milissegundos, eles são mais sensíveis às oscilações descritas na seção 4.4 e a efeitos de memória (vetores maiores deixam de caber nos níveis mais rápidos de cache do processador). Um intervalo maior reduz o efeito do ruído: de 50.000 para 500.000 (n ×10), a teoria prevê ×12,1 para Θ(n log n), e as medianas do experimento de vetores grandes foram ×12,31 para o MergeSort, ×11,71 para o QuickSort (slide) e ×12,38 para o Arrays.sort no cenário aleatório, uma concordância muito boa.

5.2 Cenário já ordenado: melhor caso × pior caso

O cenário ordenado foi o que mais evidenciou as diferenças entre os algoritmos (Tabela 5 e Figura 3). Com n = 100.000:

- O **InsertionSort** (0,064 ms) e o **BubbleSort otimizado** (0,085 ms) atingiram o melhor caso Θ(n): fizeram uma única passada pelo vetor, sem deslocamentos ou trocas.
- O **BubbleSort do slide** (3.866,24 ms) não se beneficia da ordenação: executa todas as n(n−1)/2 comparações. A diferença para a versão otimizada foi de mais de 45.000 vezes.
- Os **SelectionSort** (1.779,70 ms e 1.448,68 ms) também não se beneficiam: a busca do mínimo sempre percorre todo o restante do vetor, em qualquer cenário. Seus tempos no cenário ordenado e no aleatório foram da mesma ordem de grandeza.
- O **QuickSort do slide** (2.121,76 ms) entrou no seu pior caso, enquanto o QuickSort + shuffle levou 7,21 ms (seção 5.3).

- O **Arrays.sort** (0,046 ms) foi o mais rápido: ele detecta que o vetor já está ordenado e termina em tempo linear.

5.3 Pior caso do QuickSort e o efeito dos valores repetidos

Com o pivô fixo em `A[left]`, um vetor ordenado ou em ordem inversa produz partições extremamente desbalanceadas. Com n = 100.000, o QuickSort do slide levou 2.121,76 ms (ordenado) e 1.812,78 ms (inverso), contra 10,67 ms no aleatório. Com vetores grandes, chegou a 16.038,75 ms com n = 500.000 no ordenado e passou do limite de 20 s no inverso. O embaralhamento resolveu completamente o problema: o QuickSort + shuffle ficou entre 7 e 13 ms com n = 100.000 e entre 48 e 64 ms com n = 500.000, em todos os cenários.

Um ponto interessante é que o crescimento do pior caso foi **menor que $\Theta(n^2)$**. A explicação está nos valores repetidos. Como os valores pertencem a [1, 100.000], um vetor com n elementos tem aproximadamente $D(n) = 100.000 \cdot (1 - e^{(-n/100.000)})$ valores distintos. Na partição do slide, os elementos iguais ao pivô vão todos para a esquerda, então cada chamada remove **todas as cópias** do pivô, e não apenas um elemento. O número de níveis da recursão passa a ser $\approx D(n)$ e o custo total fica proporcional a $n \cdot D(n)$. Quando n é bem menor que 100.000, $D(n) \approx n$ e volta-se ao $\Theta(n^2)$ clássico. Quando n se aproxima ou ultrapassa 100.000, o crescimento fica mais próximo de linear. A Tabela 14 compara essa previsão com as medianas medidas.

Intervalo de n	Previsto $\Theta(n^2)$	Previsto $n \cdot D(n)$	Observado (ordenado)	Observado (inverso)
25.000 → 50.000	4,00	3,56	1,36*	3,69
50.000 → 100.000	4,00	3,21	3,16	3,25
50.000 → 150.000	9,00	5,92	6,05	6,52
150.000 → 500.000	11,11	4,26	3,94	—

* Valor discrepante: a medição com n = 25.000 no cenário ordenado ficou acima do esperado em relação às demais.

Tabela 14: razões de crescimento do QuickSort (slide) no pior caso (medianas).

A previsão que considera os valores repetidos explica muito melhor os dados do que o $\Theta(n^2)$ puro. O pior caso do QuickSort continua sendo muito mais lento que o caso médio, mas o seu crescimento depende também da distribuição dos valores, e não só de n.

5.4 Cenários inverso e parcialmente ordenado

No cenário **inverso**, todos os algoritmos quadráticos estão no pior caso: com n = 100.000, BubbleSort 6.527,60 ms, BubbleSort otimizado 6.323,61 ms (a otimização não ajuda, pois toda passada faz trocas), SelectionSort 7.993,85 ms e InsertionSort 844,13 ms. O SelectionSort estável foi o mais lento (12.436,45 ms): no inverso, o menor elemento está sempre no fim do trecho não ordenado, então cada iteração desloca todo o restante do vetor, somando cerca de $n^2/2$ escritas às comparações. Esse é o custo prático de tornar o SelectionSort estável.

No cenário **parcialmente ordenado**, o **InsertionSort** foi o grande beneficiado: 78,18 ms com n = 100.000, contra 603,03 ms no aleatório. Seu custo é proporcional ao número de inversões, que é pequeno quando só 15% das posições estão fora do lugar. Já o **BubbleSort otimizado** (3.265,32 ms)

praticamente empatou com o do slide (3.302,99 ms). Um elemento pequeno que está muito à direita da sua posição correta avança apenas uma posição para a esquerda por passada, então são necessárias muitas passadas, e o critério de parada antecipada quase nunca é atingido.

Outro efeito notável é o da **previsão de desvios** (*branch prediction*) do processador. Com n = 50.000, o BubbleSort levou 3.011,15 ms no aleatório e 1.649,86 ms no inverso, embora ambos sejam $\Theta(n^2)$ e o inverso faça mais trocas. No inverso, o resultado do `if` é sempre o mesmo e o processador acerta a previsão; no aleatório, ele erra com frequência e precisa descartar trabalho especulativo.

5.5 CountingSort

O CountingSort foi o algoritmo mais rápido na maioria das combinações, com tempos abaixo de 1,4 ms no experimento principal e de 8 ms com n = 500.000, sem grandes diferenças entre os cenários, já que ele não compara elementos. Com k = 100.000 fixo, o termo $\Theta(k)$ pesa bastante para n pequeno: de 50.000 para 500.000 (n ×10), o tempo no cenário aleatório cresceu apenas ×2,23, bem menos que 10. Os tempos abaixo de 1 ms também são os mais sujeitos a ruído relativo. A contrapartida é a memória: ele usa dois vetores auxiliares (tamanhos n e k + 1) e só é viável porque o intervalo de valores é pequeno e conhecido.

5.6 Arrays.sort × implementações próprias

Entre os algoritmos por comparação, o `Arrays.sort` foi o mais rápido ou próximo disso em todos os cenários: 9,42 ms com n = 100.000 no aleatório (QuickSort do slide: 10,67 ms; MergeSort: 16,65 ms) e 40,21 ms com n = 500.000 (QuickSort do slide: 43,18 ms; QuickSort + shuffle: 48,18 ms; MergeSort: 54,89 ms). Nos cenários ordenado e inverso, foi praticamente instantâneo (0,046 ms e 0,062 ms com n = 100.000), porque detecta trechos ordenados (inclusive decrescentes) e os aproveita. O QuickSort do slide foi um pouco mais rápido que o QuickSort + shuffle no aleatório (10,67 × 12,18 ms), o que é esperado: nesse cenário a entrada já é aleatória e o embaralhamento é custo extra. O MergeSort, embora $\Theta(n \log n)$ em todos os casos, foi mais lento que os QuickSorts no aleatório, por causa das cópias para o vetor auxiliar (constantes maiores).

5.7 Buscas

As buscas lineares cresceram aproximadamente na proporção de n. Com n triplicando (50k → 150k) e depois ×3,33 (150k → 500k), as medianas da busca linear iterativa cresceram ×4,46 e ×3,27, e as da linear de duas pontas ×3,05 e ×3,45. As buscas binárias permaneceram praticamente constantes, entre 18 e 33 ns em todos os tamanhos: $\log_2 n$ passa de 15,6 para 18,9 nesse intervalo, um aumento de apenas ~20%, da ordem da própria variação das medições. Com n = 500.000, a busca linear iterativa foi cerca de 5.000 vezes mais lenta que a binária iterativa, o que mostra na prática a diferença entre $\Theta(n)$ e $\Theta(\log n)$.

A **busca linear recursiva**, embora também $\Theta(n)$, foi de 11 a 50 vezes mais lenta que a iterativa (medianas), porque cada elemento custa uma chamada de método (criação de um quadro na pilha, passagem de parâmetros e retorno). Com n = 500.000 ela cresceu ×14,47 em relação a n = 150.000 e teve a maior variabilidade do experimento: a recursão com 500.000 níveis ocupa dezenas de megabytes de pilha, que não cabem na cache. A **linear de duas pontas** faz metade das iterações, mas

cada uma tem duas comparações, e ficou de 1,1 a 1,6 vez mais lenta que a iterativa simples. Uma provável causa é que o laço simples é mais fácil de otimizar pelo compilador JIT. Esses casos mostram que a notação assintótica descreve o crescimento, mas esconde constantes que fazem diferença na prática.

5.8 Estabilidade e uso de memória

São **estáveis** (preservam a ordem relativa de elementos iguais): BubbleSort (as duas versões), InsertionSort, MergeSort (graças ao "<=" no merge), CountingSort (graças à varredura de trás para frente) e o SelectionSort estável. **Não são estáveis** o SelectionSort do slide (no exemplo [2a, 2b, 1, 3], o 2a é trocado com o 1 e passa para depois do 2b) e o QuickSort. Em vetores de `int` a estabilidade não tem efeito visível, mas é essencial ao ordenar registros por mais de um critério, por exemplo pedidos já ordenados por data e depois ordenados por cliente.

Quanto à **memória**, MergeSort (O(n)) e CountingSort (O(n + k)) usam memória auxiliar significativa; Bubble, Selection e Insertion usam O(1). O QuickSort é *in-place*, mas sua recursão usa O(log n) de pilha em média e O(n) no pior caso. Neste trabalho, isso foi constatado na prática: foi necessário executar os experimentos em uma *thread* com pilha de 1 GB para evitar `StackOverflowError` no pior caso do QuickSort do slide e na busca linear recursiva.

5.9 Síntese das divergências entre teoria e prática

- **Constantes escondidas:** algoritmos de mesma complexidade tiveram tempos muito diferentes (linear recursiva × iterativa; SelectionSort estável × SelectionSort no inverso; MergeSort × QuickSort).
- **Previsão de desvios:** o BubbleSort foi mais lento no aleatório do que no inverso, embora faça menos trocas.
- **Valores repetidos:** o pior caso do QuickSort do slide cresceu como $n \cdot D(n)$, e não como n^2 puro (seção 5.3).
- **Efeitos de memória e cache:** razões de crescimento acima do previsto em algoritmos rápidos e grande variabilidade na busca linear recursiva com n = 500.000.
- **Variabilidade da máquina:** medições discrepantes e a interrupção do BubbleSort com n = 100.000 (seção 4.4).
- **Termos de menor ordem:** o Θ(k) do CountingSort domina para n pequeno, o que faz o tempo crescer menos que n.

6. Conclusão

Os experimentos confirmaram, em linhas gerais, as complexidades assintóticas esperadas: os algoritmos quadráticos multiplicaram seus tempos por ≈4 ao dobrar n; os algoritmos Θ(n log n) cresceram ≈12× quando n aumentou 10×; as buscas lineares cresceram proporcionalmente a n e as binárias ficaram praticamente constantes. Os cenários de entrada mostraram que o mesmo algoritmo pode ir do melhor ao pior caso (InsertionSort, BubbleSort otimizado e QuickSort do slide), e que as

diferenças mais evidentes aparecem nos cenários ordenado e inverso. As divergências observadas foram explicadas por constantes ocultas na notação O, previsão de desvios, efeitos de memória, valores repetidos na entrada e variabilidade da máquina.

Para ordenar grandes volumes de dados em uma aplicação real, os algoritmos $\Theta(n^2)$ (Bubble, Selection e Insertion) devem ser evitados, assim como o QuickSort com pivô fixo quando a entrada pode estar ordenada. Na grande maioria dos casos, a melhor escolha é o `Arrays.sort()`, que foi o mais rápido entre os algoritmos por comparação, é adaptativo a dados já ordenados e é amplamente testado (para objetos, `Arrays.sort(Object[])` e `Collections.sort` usam o TimSort, que é estável). Uma implementação própria se justifica em situações específicas, como o CountingSort quando os valores estão em um intervalo pequeno e conhecido, ou o InsertionSort para vetores pequenos ou quase ordenados.

Referências

- AGUIAR, J. J. B. Slides da disciplina Estrutura de Dados: Análise de Algoritmos Recursivos; Algoritmos de Ordenação. Campina Grande: UEPB, 2025.
- CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. Algoritmos: Teoria e Prática. 3. ed. Rio de Janeiro: Elsevier, 2012.
- ORACLE. Java Platform, Standard Edition: documentação da classe java.util.Arrays.